

ASCENSION

Linguaggio di Programmazione

Manuale Utente Completo

Versione 12.8 (Atomo Edition)

Autore: EdeFede

Licenza: GPL v3

Repository: github.com/edefede/ascension

NOTA IMPORTANTE

Ascension è un linguaggio di programmazione sviluppato per uso didattico e hobbistico. È pensato per chi vuole imparare i concetti della programmazione in modo semplice e diretto, sperimentando con una sintassi pulita e intuitiva.

Ascension E la sintassi: il vero valore sta nel design del linguaggio stesso. Attualmente esistono due implementazioni ufficiali:

- **Python** (riferimento) - `ascension_12_8.py`
- **C++** (ad alte prestazioni) - header-only, modulare

Entrambe le implementazioni producono lo stesso comportamento, garantendo che il codice Ascension sia portabile tra le due versioni.

Indice

- * 1. Introduzione
- * 2. Installazione e Utilizzo
- * 3. Sintassi di Base
- * 4. Tipi di Dati
- * 5. Operatori
- * 6. Strutture di Controllo
- * 7. Funzioni
- * 8. Strutture Dati (Struct)
- * 9. Array e Matrici
- * 10. Gestione File
- * 11. Input/Output
- * 12. Funzioni Matematiche (v12.7+)
- * 13. Gestione Errori (Try/Catch)
- * 14. Networking (HTTP)
- * 15. Socket TCP
- * 16. Interfaccia Grafica (Tkinter)
- * 17. Interfaccia Terminale (Curses)
- * 18. Comandi di Sistema
- * 19. Include e Modularità
- * 20. Funzioni Built-in Complete
- * 21. Esempi Pratici
- * 22. Appendice: Riferimento Rapido

1. Introduzione

Ascension è un linguaggio di programmazione moderno con sintassi C-style, progettato per essere semplice da imparare ma sufficientemente potente per creare applicazioni reali. È implementato come una macchina virtuale stack-based con un compilatore che traduce il codice sorgente in bytecode.

Implementazioni Disponibili

Ascension dispone di due implementazioni ufficiali che garantiscono lo stesso comportamento:

Python (Riferimento) - File singolo `ascension_12_8.py`, facile da modificare e studiare. Richiede Python 3.6+ e opzionalmente le librerie `'requests'` e `'tkinter'`.

C++ (Alte Prestazioni) - Architettura header-only modulare. Compilazione con flag opzionali (`-DHAS_CURSES`, `-DHAS_NETWORK`) per abilitare moduli specifici. Ideale per ambienti embedded o quando serve maggiore velocità.

Caratteristiche Principali

- * Sintassi pulita ispirata al C, senza dichiarazioni di tipo esplicite
- * Macchina virtuale stack-based efficiente
- * Supporto per strutture dati (struct) personalizzate
- * Array monodimensionali e matrici bidimensionali
- * Funzioni con scope locale e supporto per ricorsione
- * Funzioni matematiche complete (v12.7+)
- * Gestione eccezioni con try/catch
- * I/O su file completo
- * Networking HTTP (GET/POST) e Socket TCP
- * Interfaccia grafica tramite Tkinter
- * Interfaccia terminale tramite Curses
- * Esecuzione comandi di sistema
- * Sistema di include per codice modulare

2. Installazione e Utilizzo

Download da GitHub

Ascension è disponibile su GitHub. Per scaricarlo:

```
git clone https://github.com/edefede/ascension.git
cd ascension
```

Requisiti per Python

- * Python 3.6 o superiore
- * **requests** (opzionale) - per funzionalità HTTP
- * **tkinter** (opzionale) - per GUI (solitamente incluso)

Esecuzione con Python

```
python ascension_12_8.py programma.asc
python ascension_12_8.py programma.asc -debug
```

Il flag `-debug` visualizza il bytecode generato, utile per capire come funziona il compilatore.

Compilazione C++

```
# Compilazione base
g++ -std=c++17 -O2 ascension.cpp -o ascension

# Con supporto Curses
g++ -std=c++17 -O2 -DHAS_CURSES ascension.cpp -o ascension -lcurses

# Con supporto Network
g++ -std=c++17 -O2 -DHAS_NETWORK ascension.cpp -o ascension
```

Primo Programma

Crea un file chiamato `hello.asc`:

```
// Il classico Hello World in Ascension
// L'indentazione è opzionale ma consigliata per leggibilità

print("Ciao, Mondo!");
print("Benvenuto in Ascension 12.8!");
```

Nota sullo stile: L'indentazione in Ascension è opzionale - il compilatore la ignora. Tuttavia, è fortemente consigliata per migliorare la leggibilità del codice, specialmente con blocchi annidati.

3. Sintassi di Base

Commenti

Ascension supporta commenti su singola riga e multi-riga:

```
// Questo e un commento su singola riga

/* Questo e un commento
su piu righe - utile per
documentare funzioni */
```

Variabili

Le variabili non richiedono dichiarazione di tipo. Il tipo e determinato automaticamente dal valore assegnato:

```
// Esempi di variabili con diversi tipi
nome = "Mario"; // Stringa
eta = 25; // Intero
altezza = 1.75; // Float
attivo = true; // Booleano
risultato = NULL; // Valore nullo
```

Variabili Globali

Per modificare una variabile globale dall'interno di una funzione, usa la keyword 'global':

```
contatore = 0; // Variabile globale

func incrementa() {
    global contatore = contatore + 1;
}

incrementa();
incrementa();
print(contatore); // Output: 2
```

Statement e Blocchi

Ogni istruzione termina con punto e virgola. I blocchi sono delimitati da graffe:

```
// L'indentazione e opzionale ma rende il codice leggibile
x = 10;

if (x > 5) {
    print("x e maggiore di 5");
    print("Valore: " + x);
}
```

4. Tipi di Dati

Tipi Primitivi

Tipo	Esempio	Descrizione
Intero	42, -17, 0	Numeri interi
Float	3.14, -0.5	Numeri decimali
Stringa	"Hello"	Testo tra virgolette
Booleano	true, false	Valori logici (1 e 0)
NULL	NULL	Assenza di valore

Stringhe e Sequenze di Escape

```
// Sequenze di escape supportate
multiriga = "Prima riga\nSeconda riga";
tabulato = "Col1\tCol2\tCol3";
con_quote = "Disse: \"Ciao!\"";
con_backslash = "Path: C:\\Users\\Mario";

print(multiriga);
// Output:
// Prima riga
// Seconda riga
```

NULL - Valore Speciale

NULL indica assenza di valore o errore. E' diverso da 0 e dalla stringa vuota:

```
// NULL e' utile per gestire errori
file = open("inesistente.txt", "r");

if (file == NULL) {
    print("Errore: file non trovato!");
} else {
    contenuto = read_all(file);
    close(file);
}
```

5. Operatori

Operatori Aritmetici

Operatore	Descrizione	Esempio
+	Addizione / Concatenazione	5 + 3 = 8, "Hello" + " World"
-	Sottrazione	10 - 4 = 6
*	Moltiplicazione	6 * 7 = 42
/	Divisione	15 / 3 = 5
%	Modulo (resto)	17 % 5 = 2

Operatori di Confronto

Operatore	Descrizione	Esempio
==	Uguale	x == 10
!=	Diverso	x != 0
>	Maggiore	x > 5
<	Minore	x < 100
>=	Maggiore o uguale	x >= 0
<=	Minore o uguale	x <= 50

Operatori Logici e Composti

```
// Operatori logici
if (eta >= 18 && eta <= 65) {
  print("Eta lavorativa");
}

if (ruolo == "admin" || ruolo == "moderatore") {
  print("Accesso consentito");
}

if (!trovato) {
  print("Elemento non trovato");
}

// Operatori composti di assegnamento
x = 10;
x += 5; // x = 15
x -= 3; // x = 12
x *= 2; // x = 24
x /= 4; // x = 6
```

6. Strutture di Controllo

If / Else If / Else

```
// Esempio completo con if/else if/else
voto = 75;

if (voto >= 90) {
    print("Eccellente!");
    print("Complimenti!");
} else if (voto >= 70) {
    print("Buono");
} else if (voto >= 60) {
    print("Sufficiente");
} else {
    print("Insufficiente");
    print("Devi studiare di piu!");
}
```

While

```
// Countdown con while
countdown = 5;

while (countdown > 0) {
    print("Mancano " + countdown + " secondi...");
    countdown -= 1;
}

print("PARTENZA!");
```

For

```
// Ciclo for classico C-style
print("Tabellina del 7:");

for (i = 1; i <= 10; i += 1) {
    risultato = 7 * i;
    print("7 x " + i + " = " + risultato);
}
```

Switch

```
// Menu con switch
print("1. Nuovo gioco");
print("2. Carica partita");
print("3. Opzioni");
print("4. Esci");

scelta = to_int(read());

switch (scelta) {
    case 1: {
        print("Avvio nuovo gioco...");
    };
    case 2: {
        print("Caricamento partita...");
    };
}
```



```
case 3: {
print("Apertura opzioni...");
};
case 4: {
print("Arrivederci!");
};
default: {
print("Scelta non valida!");
};
}
```

Break e Continue

```
// Cerca il primo numero divisibile per 7
for (i = 1; i <= 100; i += 1) {
if (i % 7 == 0) {
print("Trovato: " + i);
break; // Esce dal loop
}
}

// Stampa solo i numeri dispari
for (i = 1; i <= 10; i += 1) {
if (i % 2 == 0) {
continue; // Salta i pari
}
print(i);
}
```

7. Funzioni

Definizione e Chiamata

```
// Funzione semplice
func saluta(nome) {
    print("Ciao, " + nome + "!");
    print("Benvenuto in Ascension!");
}

saluta("Mario");
saluta("Luigi");
```

Funzioni con Valore di Ritorno

```
// Funzione che calcola l'area di un rettangolo
func area_rettangolo(base, altezza) {
    area = base * altezza;
    return area;
}

// Funzione che verifica se un numero e primo
func e_primo(n) {
    if (n < 2) {
        return false;
    }

    for (i = 2; i * i <= n; i += 1) {
        if (n % i == 0) {
            return false;
        }
    }
    return true;
}

// Uso delle funzioni
print("Area: " + area_rettangolo(5, 3));

if (e_primo(17)) {
    print("17 e primo!");
}
```

Ricorsione

```
// Fattoriale ricorsivo
func fattoriale(n) {
    if (n <= 1) {
        return 1;
    }
    return n * fattoriale(n - 1);
}

// Fibonacci ricorsivo
func fibonacci(n) {
    if (n <= 1) {
        return n;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}
```

```
}  
  
print("5! = " + fattoriale(5)); // 120  
print("Fib(10) = " + fibonacci(10)); // 55
```

Prototipi (Forward Declarations)

I prototipi permettono la ricorsione mutua tra funzioni:

```
// Dichiarazione prototipi  
func e_pari(n);  
func e_dispari(n);  
  
// Implementazione  
func e_pari(n) {  
    if (n == 0) {  
        return true;  
    }  
    return e_dispari(n - 1);  
}  
  
func e_dispari(n) {  
    if (n == 0) {  
        return false;  
    }  
    return e_pari(n - 1);  
}  
  
print(e_pari(10)); // 1 (true)  
print(e_dispari(10)); // 0 (false)
```

8. Strutture Dati (Struct)

Definizione e Uso

```
// Definizione di una struttura
struct Persona {
    nome,
    eta,
    citta
}

// Creazione e uso di istanze
mario = new Persona();
mario.nome = "Mario Rossi";
mario.eta = 30;
mario.citta = "Roma";

luigi = new Persona();
luigi.nome = "Luigi Verdi";
luigi.eta = 25;
luigi.citta = "Milano";

print(mario.nome + " ha " + mario.eta + " anni");
print("Vive a " + mario.citta);
```

Struct Annidati

```
// Strutture piu complesse
struct Indirizzo {
    via,
    numero,
    cap,
    citta
}

struct Cliente {
    nome,
    email,
    indirizzo
}

// Creazione con struct annidato
cliente = new Cliente();
cliente.nome = "Anna Bianchi";
cliente.email = "anna@email.com";
cliente.indirizzo = new Indirizzo();
cliente.indirizzo.via = "Via Roma";
cliente.indirizzo.numero = 42;
cliente.indirizzo.cap = "00100";
cliente.indirizzo.citta = "Roma";

print(cliente.nome);
print(cliente.indirizzo.via + ", " + cliente.indirizzo.numero);
```

Dizionari Inline

```
// Dizionario vuoto
config = {};
```

```
config.debug = true;
config.version = "1.0";

// Dizionario con valori iniziali
punto = {x: 10, y: 20};
print("Punto: (" + punto.x + ", " + punto.y + ")");

// Dizionario per configurazione
settings = {
volume: 80,
luminosita: 100,
fullscreen: false
};
```

9. Array e Matrici

Array Monodimensionali

```
// Creazione e popolamento
nomi[0] = "Alice";
nomi[1] = "Bob";
nomi[2] = "Charlie";
nomi[3] = "Diana";

// Iterazione con for
print("Lista partecipanti:");
for (i = 0; i < 4; i += 1) {
    print((i + 1) + ". " + nomi[i]);
}

// Funzioni utili
lunghezza = len(nomi);
print("Totale: " + lunghezza + " partecipanti");
```

Matrici 2D

```
// Due sintassi equivalenti per matrici

// Sintassi C-style
griglia[0][0] = "X";
griglia[0][1] = "O";
griglia[1][0] = "O";
griglia[1][1] = "X";

// Sintassi con virgola
griglia[0, 0] = "X";
griglia[0, 1] = "O";

// Stampa griglia
for (r = 0; r < 2; r += 1) {
    riga = "";
    for (c = 0; c < 2; c += 1) {
        riga = riga + griglia[r][c] + " ";
    }
    print(riga);
}
```

Funzione matrix()

```
// Crea matrice inizializzata
m = matrix(3, 4, 0); // 3 righe, 4 colonne, valore 0

// Riempi con valori
for (r = 0; r < rows(m); r += 1) {
    for (c = 0; c < cols(m); c += 1) {
        m[r, c] = r * cols(m) + c;
    }
}

// Info sulla matrice
print("Righe: " + rows(m)); // 3
print("Colonne: " + cols(m)); // 4
```

```
print("Dimensioni: " + dim(m)); // 2 (bidimensionale)
```

10. Gestione File

Modalita di apertura: "r" (lettura), "w" (scrittura), "a" (append)

Scrittura su File

```
// Scrittura di un file di log
log = open("app.log", "w");

if (log != NULL) {
write(log, "=== Log Applicazione ===\n");
write(log, "Avvio: OK\n");
write(log, "Versione: 1.0\n");
close(log);
print("Log creato con successo!");
} else {
print("Errore nella creazione del log");
}
```

Lettura da File

```
// Lettura completa
f = open("documento.txt", "r");

if (f != NULL) {
contenuto = read_all(f);
print(contenuto);
close(f);
}

// Lettura riga per riga
f = open("lista.txt", "r");

if (f != NULL) {
numero_riga = 1;
linea = read_line(f);

while (len(linea) > 0) {
print(numero_riga + ": " + linea);
numero_riga += 1;
linea = read_line(f);
}
close(f);
}
```

Funzioni File I/O

Funzione	Descrizione	Ritorno
open(file, modo)	Apre un file	handle o NULL
write(h, testo)	Scrive nel file	1 o NULL
read_line(h)	Legge una riga	stringa o NULL
read_all(h)	Legge tutto	stringa o NULL
close(h)	Chiude il file	1 o NULL

11. Input/Output

Output con print()

```
// Varie forme di output
print("Messaggio semplice");

nome = "Mario";
punti = 1500;
print("Giocatore:", nome, "- Punti:", punti);

// Concatenazione per formattazione
print("Ciao " + nome + "! Hai totalizzato " + punti + " punti.");
```

Input con read()

```
// Input interattivo
print("=== Registrazione ===");

print("Nome:");
nome = read();

print("Eta:");
eta = to_int(read());

print("Email:");
email = read();

print("");
print("Riepilogo:");
print("Nome: " + nome);
print("Eta: " + eta + " anni");
print("Email: " + email);
```

Conversioni di Tipo

```
// Conversioni disponibili
stringa_num = "42";
numero = to_int(stringa_num); // 42 (intero)

stringa_dec = "3.14159";
decimale = to_float(stringa_dec); // 3.14159 (float)

// Caratteri e codici ASCII
codice = to_int("A"); // 65
carattere = chr(65); // "A"
carattere = chr(10); // newline

// Sottostringa
testo = "Hello, World!";
saluto = substr(testo, 0, 5); // "Hello"
mondo = substr(testo, 7, 5); // "World"
```

12. Funzioni Matematiche (v12.7+)

Dalla versione 12.7 (Math Edition), Ascension include un set completo di funzioni matematiche built-in.

Numeri Casuali

```
// Tre forme di random()
r = random(); // Float in [0.0, 1.0)
dato = random(6) + 1; // Intero in [1, 6]
n = random(10, 100); // Intero in [10, 100)

// Esempio: simula lancio dadi
print("Lancio due dadi:");
dato1 = random(6) + 1;
dato2 = random(6) + 1;
print("Dado 1: " + dato1);
print("Dado 2: " + dato2);
print("Totale: " + (dato1 + dato2));
```

Funzioni Base

Funzione	Descrizione	Esempio
sqrt(x)	Radice quadrata	sqrt(16) = 4
pow(b, e)	Potenza	pow(2, 10) = 1024
exp(x)	Esponenziale e^x	exp(1) = 2.718...
log(x)	Logaritmo naturale	log(E) = 1
abs(x)	Valore assoluto	abs(-42) = 42
floor(x)	Arrotonda per difetto	floor(3.7) = 3
ceil(x)	Arrotonda per eccesso	ceil(3.2) = 4

Funzioni Trigonometriche

Tutte le funzioni trigonometriche lavorano in radianti:

```
// Trigonometria base
angolo = PI / 4; // 45 gradi
print("sin(45) = " + sin(angolo));
print("cos(45) = " + cos(angolo));
print("tan(45) = " + tan(angolo));

// Funzioni inverse
valore = 0.5;
print("asin(0.5) = " + asin(valore));
print("acos(0.5) = " + acos(valore));

// atan2 per angoli da coordinate
x = 1;
y = 1;
angolo = atan2(y, x); // PI/4
```

Costanti e Esempio Pratico

```
// Costanti disponibili
print("PI = " + PI); // 3.14159...
print("E = " + E); // 2.71828...

// Esempio: calcola distanza tra due punti
func distanza(x1, y1, x2, y2) {
  dx = x2 - x1;
  dy = y2 - y1;
  return sqrt(dx*dx + dy*dy);
}

// Esempio: calcola area cerchio
func area_cerchio(raggio) {
  return PI * raggio * raggio;
}

print("Distanza (0,0)-(3,4) = " + distanza(0, 0, 3, 4));
print("Area cerchio r=5 = " + area_cerchio(5));
```

13. Gestione Errori (Try/Catch)

Try/Catch

```
// Gestione errori con try/catch
try {
    print("Tento operazione rischiosa...");
    risultato = 10 / 0; // Errore!
    print("Questo non viene eseguito");
} catch (errore) {
    print("Errore catturato: " + errore);
    print("Il programma continua...");
}

print("Fine del programma");
```

Throw - Eccezioni Personalizzate

```
// Funzione con validazione
func dividi_sicuro(a, b) {
    if (b == 0) {
        throw "Errore: divisione per zero!";
    }
    return a / b;
}

func radice_sicura(n) {
    if (n < 0) {
        throw "Errore: radice di numero negativo!";
    }
    return sqrt(n);
}

// Uso con gestione errori
try {
    r1 = dividi_sicuro(10, 2); // OK: 5
    print("10 / 2 = " + r1);

    r2 = dividi_sicuro(10, 0); // Lancia eccezione
    print("Questo non appare");
} catch (e) {
    print(e);
}
```

14. Networking (HTTP)

Richiede la libreria Python 'requests'

HTTP GET

```
// Richiesta GET semplice
risposta = http_get("https://api.example.com/users");

status = response_status(risposta);
body = response_body(risposta);

if (status == 200) {
    print("Risposta ricevuta:");
    print(body);
} else {
    print("Errore HTTP: " + status);
}
```

HTTP POST

```
// Invio dati con POST
dati = {
    nome: "Mario Rossi",
    email: "mario@example.com",
    eta: 30
};

risposta = http_post("https://api.example.com/users", dati);

if (response_status(risposta) == 201) {
    print("Utente creato con successo!");
    print(response_body(risposta));
}
```

15. Socket TCP

Client TCP

```
// Client TCP semplice
ip = get_ip("example.com"); // Risoluzione DNS
print("IP: " + ip);

sock = socket_open("TCP", "IPv4");

if (sock != NULL) {
    if (socket_connect(sock, ip, 80) != NULL) {
        // Invia richiesta HTTP
        request = "GET / HTTP/1.1\r\n";
        request = request + "Host: example.com\r\n";
        request = request + "\r\n";

        socket_send(sock, request);
        risposta = socket_recv(sock, 4096);
        print(risposta);
    }
}
```

```
socket_close(sock);  
}
```

16. Interfaccia Grafica (Tkinter)

Richiede tkinter installato in Python

Applicazione GUI Completa

```
// Contatore con interfaccia grafica
global contatore = 0;

func aggiorna_label() {
tk_config(label, {text: "Contatore: " + contatore});
}

func incrementa() {
global contatore = contatore + 1;
aggiorna_label();
}

func decrementa() {
global contatore = contatore - 1;
aggiorna_label();
}

func reset() {
global contatore = 0;
aggiorna_label();
}

// Creazione interfaccia
root = tk_root("Contatore");
tk_geometry("300x150");

label = tk_widget(root, "Label", {text: "Contatore: 0", font: "Arial 18"});
tk_pack(label, {pady: 20});

frame = tk_widget(root, "Frame", {});
tk_pack(frame, {});

btn_meno = tk_widget(frame, "Button", {text: " - ", width: 5});
tk_command(btn_meno, "decrementa");
tk_pack(btn_meno, {side: "left", padx: 5});

btn_reset = tk_widget(frame, "Button", {text: "Reset", width: 5});
tk_command(btn_reset, "reset");
tk_pack(btn_reset, {side: "left", padx: 5});

btn_piu = tk_widget(frame, "Button", {text: " + ", width: 5});
tk_command(btn_piu, "incrementa");
tk_pack(btn_piu, {side: "left", padx: 5});

tk_mainloop();
```

Widget disponibili: Label, Button, Entry, Text, Listbox, Canvas, Frame

17. Interfaccia Terminale (Curses)

Curses permette di creare interfacce testuali interattive nel terminale.

Esempio con Dimensioni Terminale (v12.8+)

```
// Editor status bar con dimensioni dinamiche
curses_init();
curses_clear();

// Ottieni dimensioni terminale
righe = curses_rows();
colonne = curses_cols();

// Disegna bordo superiore
curses_move(0, 0);
for (i = 0; i < colonne; i += 1) {
    curses_write("=");
}

// Titolo centrato
titolo = "ASCENSION EDITOR v1.0";
centro_x = (colonne - len(titolo)) / 2;
curses_move(1, centro_x);
curses_write(titolo);

// Status bar in fondo
curses_move(righe - 1, 0);
status = "Terminale: " + colonne + "x" + righe + " | Premi Q per uscire";
curses_write(status);

curses_refresh();

// Loop principale
running = true;
while (running) {
    tasto = curses_read_key();
    if (tasto == 113) { // 'q'
        running = false;
    }
}

curses_end();
```

Funzioni Curses

Funzione	Descrizione
curses_init()	Inizializza modalita curses
curses_end()	Termina e ripristina terminale
curses_clear()	Pulisce lo schermo
curses_refresh()	Aggiorna il display
curses_move(y, x)	Sposta il cursore
curses_write(testo)	Scrive alla posizione corrente
curses_read_key()	Attende e legge un tasto

curses_rows()	Righe del terminale (v12.8+)
curses_cols()	Colonne del terminale (v12.8+)

18. Comandi di Sistema

```
// Esegui comando e ottieni exit code
exitCode = system("mkdir nuovo_progetto");

if (exitCode == 0) {
    print("Directory creata!");
} else {
    print("Errore nella creazione");
}

// Esegui comando e cattura output
data_ora = exec("date");
print("Data e ora: " + data_ora);

// Lista file nella directory corrente
files = exec("ls -la");
print(files);
```

Attenzione: L'esecuzione di comandi di sistema può essere pericolosa. Non usare mai con input non validato dall'utente.

19. Include e Modularità

La direttiva include permette di organizzare il codice in moduli:

```
// File: utils.asc
func stampa_separatore() {
    print("=====");
}

func stampa_titolo(titolo) {
    stampa_separatore();
    print(" " + titolo);
    stampa_separatore();
}

// File: main.asc
include "utils.asc";

stampa_titolo("Il Mio Programma");
print("Contenuto del programma...");
```

Organizzazione Consigliata

```
progetto/
+-- main.asc // Punto di ingresso
+-- lib/
| +-- strings.asc // Funzioni stringhe
| +-- array.asc // Funzioni array
| +-- math_ext.asc // Funzioni matematiche estese
+-- config.asc // Configurazioni
```

Librerie standard disponibili: strings.asc, array.asc, math_ext.asc - scaricabili dal repository GitHub.

20. Funzioni Built-in Complete

I/O e Conversioni

`print(args...)`, `read()`, `to_int(v)`, `to_float(v)`, `chr(code)`

Stringhe e Array

`len(obj)`, `keys(arr)`, `substr(str, start, len)`

Matrici

`matrix(rows, cols, init)`, `rows(m)`, `cols(m)`, `dim(arr)`

Matematica (v12.7+)

`random()`, `random(max)`, `random(min, max)`
`sqrt(x)`, `pow(base, exp)`, `exp(x)`, `log(x)`
`abs(x)`, `floor(x)`, `ceil(x)`
`sin(x)`, `cos(x)`, `tan(x)`, `asin(x)`, `acos(x)`, `atan(x)`, `atan2(y, x)`
Costanti: `PI`, `E`

File I/O

`open(file, mode)`, `write(h, text)`, `read_line(h)`, `read_all(h)`, `close(h)`

HTTP

`http_get(url)`, `http_post(url, data)`, `response_status(r)`, `response_body(r)`

Socket

`socket_open(type, proto)`, `socket_bind/listen/accept/connect/send/rcv/close`
`get_ip(hostname)`

GUI (Tkinter)

`tk_root`, `tk_widget`, `tk_pack`, `tk_grid`, `tk_config`, `tk_command`, `tk_bind`
`tk_get`, `tk_set`, `tk_clear`, `tk_focus`, `tk_destroy`
`tk_msgbox`, `tk_askstring`, `tk_askyesno`, `tk_filedialog_open/save`
`tk_canvas_line/rect/oval/text/clear/delete/move`
`tk_after`, `tk_after_cancel`, `tk_update`, `tk_mainloop`

Curses (TUI)

`curses_init`, `curses_end`, `curses_clear`, `curses_refresh`
`curses_move(y, x)`, `curses_write(text)`, `curses_read_key()`
`curses_rows()`, `curses_cols()` (v12.8+)

Sistema

`system(cmd)` - ritorna exit code, `exec(cmd)` - ritorna output

21. Esempi Pratici

Gioco: Indovina il Numero

```
// Gioco indovina il numero
print("=== INDOVINA IL NUMERO ===");
print("Ho pensato un numero da 1 a 100.");
print("");

segreto = random(100) + 1;
tentativi = 0;
indovinato = false;

while (!indovinato) {
    print("Inserisci il tuo tentativo:");
    guess = to_int(read());
    tentativi += 1;

    if (guess < segreto) {
        print("Troppo basso! Riprova.");
    } else if (guess > segreto) {
        print("Troppo alto! Riprova.");
    } else {
        indovinato = true;
        print("");
        print("ESATTO! Hai indovinato in " + tentativi + " tentativi!");
    }
}
```

Calcolatrice Scientifica

```
// Calcolatrice con funzioni matematiche
print("=== CALCOLATRICE SCIENTIFICA ===");
print("Operazioni: +, -, *, /, sqrt, pow, sin, cos");
print("");

print("Primo numero:");
a = to_float(read());

print("Operazione:");
op = read();

switch (op) {
    case "sqrt": {
        print("sqrt(" + a + ") = " + sqrt(a));
    };
    case "sin": {
        print("sin(" + a + ") = " + sin(a));
    };
    case "cos": {
        print("cos(" + a + ") = " + cos(a));
    };
    default: {
        print("Secondo numero:");
        b = to_float(read());

        switch (op) {
            case "+": { print(a + " + " + b + " = " + (a + b)); };
        }
    }
}
```

```
case "-": { print(a + " - " + b + " = " + (a - b)); };  
case "*": { print(a + " * " + b + " = " + (a * b)); };  
case "/": { print(a + " / " + b + " = " + (a / b)); };  
case "pow": { print(a + "^" + b + " = " + pow(a, b)); };  
};  
};  
}
```

Crivello di Eratostene

```
// Trova tutti i numeri primi fino a N
print("Crivello di Eratostene");
print("Inserisci il limite:");
limite = to_int(read());

// Inizializza array (true = potenzialmente primo)
for (i = 0; i <= limite; i += 1) {
    is_primo[i] = true;
}

is_primo[0] = false;
is_primo[1] = false;

// Applica il crivello
for (i = 2; i * i <= limite; i += 1) {
    if (is_primo[i]) {
        // Marca tutti i multipli come non primi
        for (j = i * i; j <= limite; j += i) {
            is_primo[j] = false;
        }
    }
}

// Stampa i primi trovati
print("Numeri primi fino a " + limite + ":");
conta = 0;
for (i = 2; i <= limite; i += 1) {
    if (is_primo[i]) {
        print(i);
        conta += 1;
    }
}
print("Totale: " + conta + " primi");
```

22. Appendice: Riferimento Rapido

Sintassi Essenziale

```
// Variabili
x = 10;
global y = 20;

// Funzioni
func nome(a, b) {
return a + b;
}

// Struct
struct Nome { campo1, campo2 }
obj = new Nome();

// Controllo flusso
if (cond) { ... } else if (cond2) { ... } else { ... }
for (i = 0; i < n; i += 1) { ... }
while (cond) { ... }
switch (val) { case 1: { ... }; default: { ... }; }

// Eccezioni
try { ... } catch (e) { ... }
throw "errore";

// Include
include "file.asc";
```

Operatori

```
Aritmetici: + - * / %
Confronto: == != > < >= <=
Logici: && || !
Assegnam.: = += -= *= /=
```

Valori Speciali

```
true - booleano vero (1)
false - booleano falso (0)
NULL - assenza di valore
PI - 3.14159... (v12.7+)
E - 2.71828... (v12.7+)
```

Download e Risorse

```
git clone https://github.com/edefede/ascension.git
cd ascension
```

Ascension v12.8 (Atomo Edition)

Implementazioni: Python (riferimento) e C++ (alte prestazioni)

Autore: EdeFede | Licenza: GPL v3 | github.com/edefede/ascension

Ricorda: **Ascension E la sintassi**. L'implementazione puo cambiare, ma l'essenza del linguaggio rimane.